System Design Document

CSE 362

Software Development and Professional Practice

Winter 2024

© Royal Military College of Canada

SDD – "System Design Document"

- The RAD is usually a one-shot deal, viewed and approved of by the client.

- The SDD is for the development team only – it is not shown to the client. It is technical – more diagrams than words.

- It should be a dynamic document and is really just a framework that evolves or is filled in as the design of your project matures.

SDD as a Deliverable

- You do not have to use all categories on the next two slides,

but need to have put what analysis results you have (e.g. UML diagrams) in the right places.

SDD - Traditional Contents

- Create something like this table of contents as a framework to hold your design information:

1. Introduction
    1. Purpose of system
    2. Design goals
    3. Definition, acronyms and abbreviations
    4. References
    5. Overview
2. Current software architecture (*not needed for us*)

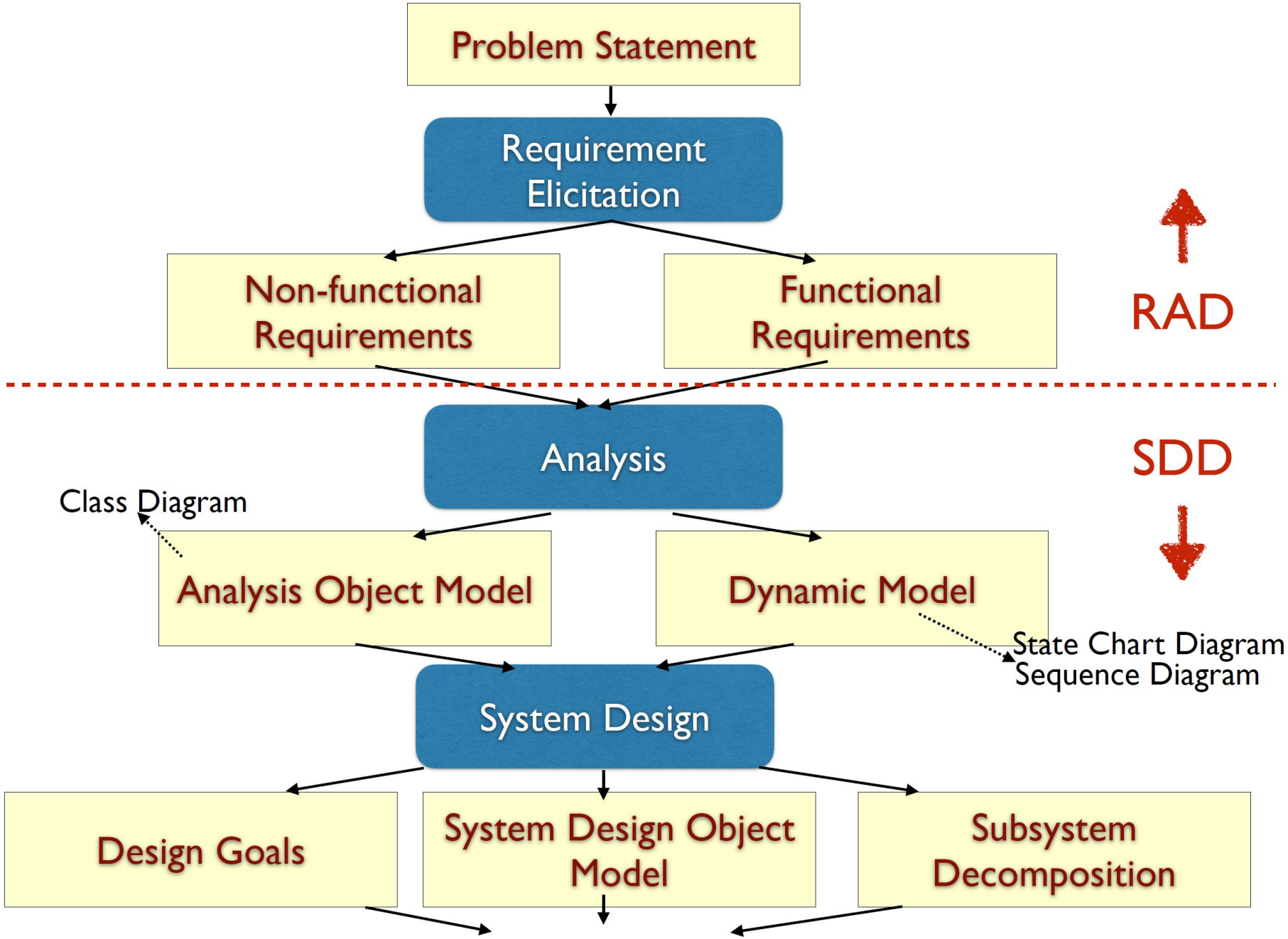
SDD Contents, Cont.

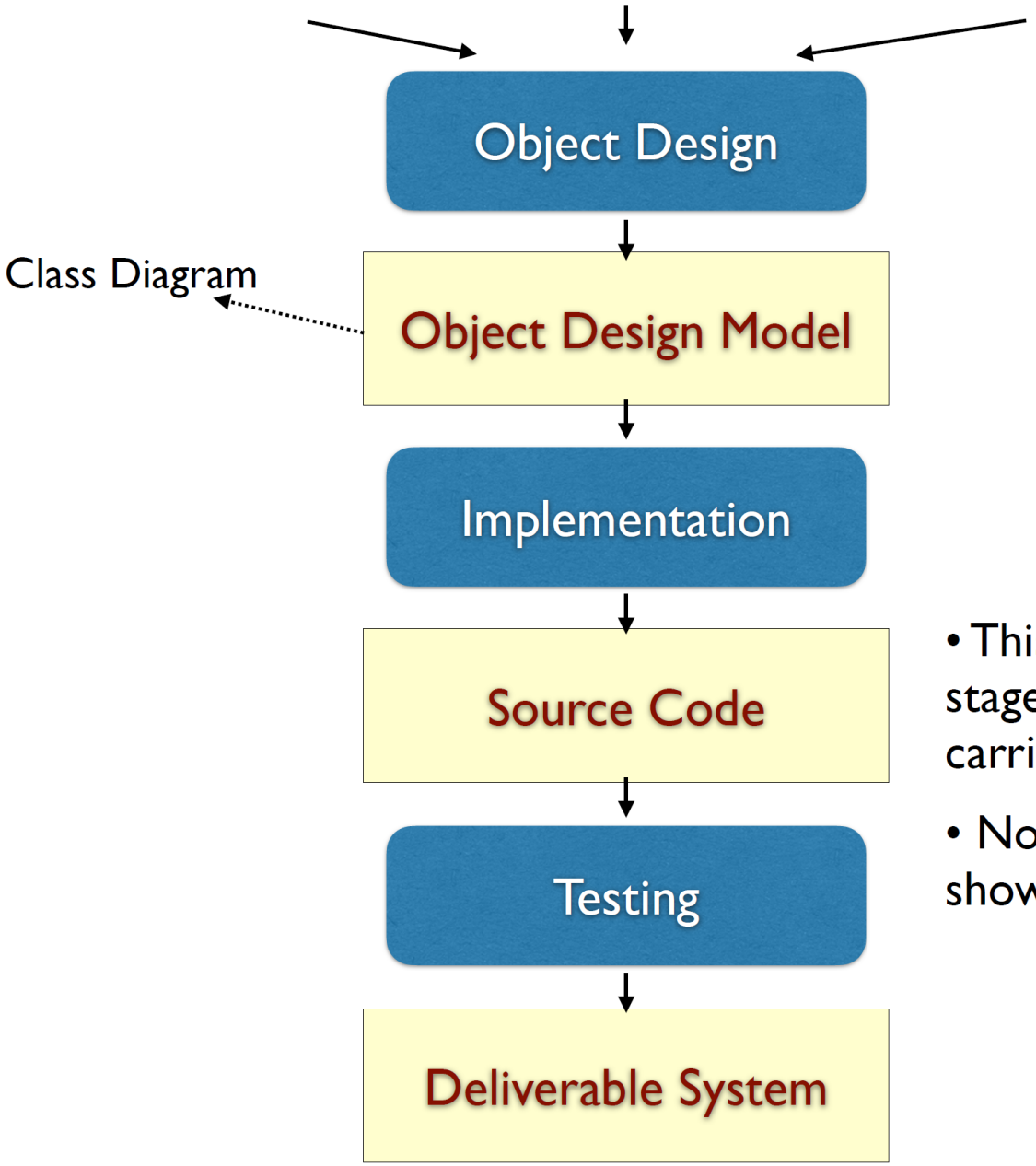
3. Proposed software architecture
    1. Overview
    2. Subsystem decomposition
    3. Hardware/software mapping
    4. Persistent data management
    5. Access control and security
    6. Global flow control
    7. Boundary conditions
4. Subsystem services
5. Glossary

SDD Contents: Extra guidelines

- At a minimum, the SDD should contain a use case diagram, a class diagram, as many sequence diagrams as needed to describe the dynamic operation of your software, and a state diagram.
- Screen captures of the GUI windows you have constructed. You can build the GUI without adding all the necessary functionality.
- A list of specific coding assignments. Who is writing which classes/features? Who is responsible for the GUI construction? Who is going to do the testing?
- A detailed timeline with specific milestones for these tasks, with a completion in week 13. Consider the order of construction carefully and start testing as early on as possible.

Software Design


Problem Statement
Requirement Elicitation
Non-functional Requirements
Functional Requirements
RAD
Analysis
SDD
Class Diagram
Analysis Object Model
Dynamic Model
State Chart Diagram
Sequence Diagram
System Design
Design Goals
System Design Object Model
Subsystem Decomposition



- This is not a flowchart. The stages are not necessarily carried out in this order.
- No iteration or feedback is shown.

After Software Analysis

Analysis has provided:

- *A set of Functional and Non-functional Requirements and Constraints*

- *A System Model, expressed in:*
  - *Use Cases*
  - *Object Models*
  - *Sequence Diagrams*
  - *Statechart Diagrams*
  - …

After System Design

End up with:

- Design Goals

– *System qualities that need to be optimized*

- Software Architecture

– *Subsystem decomposed, mapped to hardware, decisions such as control flow, access control and data storage made.*

- Boundary Use Cases

– *System configuration, startup, shutdown and exception handling.*

Subsystem Decomposition

- The aim is to identify and/or create classes (or packages of classes) that can be coded by a single coder.

- Techniques used are Layering and Partitioning.
- Concerned about Coupling between subsystems and Cohesion within a subsystem.

- The Subsystems must address the design goals. Keep refining the Decomposition until the goals are satisfied.

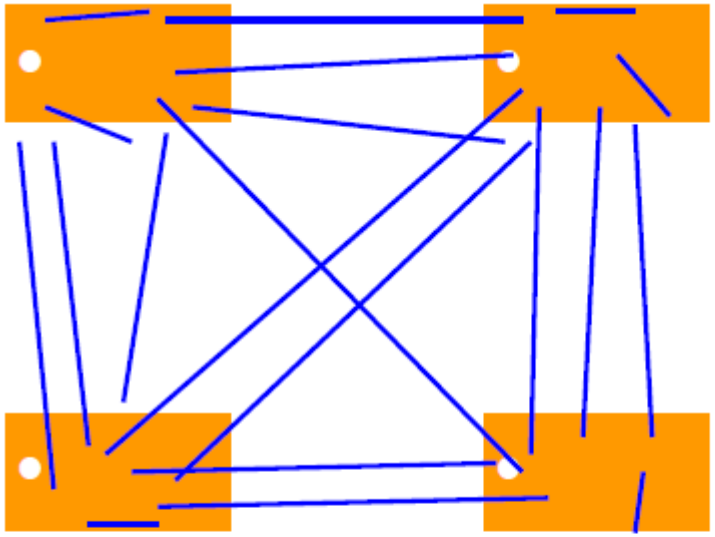
Identifying Subsystems

- A subsystem is like a package in Java – a group of classes related by their functionality.

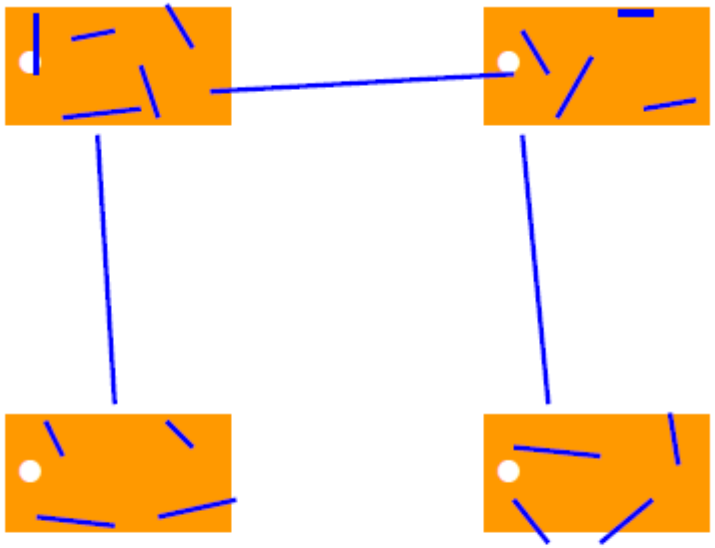
- Like classes in a Java package, objects in a subsystem will have some coupling within the subsystem, but should have minimal coupling with other subsystems.

- What is *coupling* anyway?

Coupling and Cohesion



Strong Coupling, Weak Cohesion



Strong Cohesion, Weak Coupling

Coupling and Cohesion, Cont.

- Coupling occurs between packages and cohesion occurs between methods.

- Weak coupling allows you to make changes in one object and minimize the effects on other objects.
- Strong cohesion means that methods are small and single-purpose.

Layering

A hierarchical decomposition

- A **layer** is a grouping of subsystems providing related services to a higher level of abstraction
- a layer depends on lower layers
- a layer has no knowledge of higher layers

- *Closed architecture* -- a layer depends only one the one immediately below it
- *Open architecture* -- a layer can access the one immediately below, as well as deeper layers

Partitioning

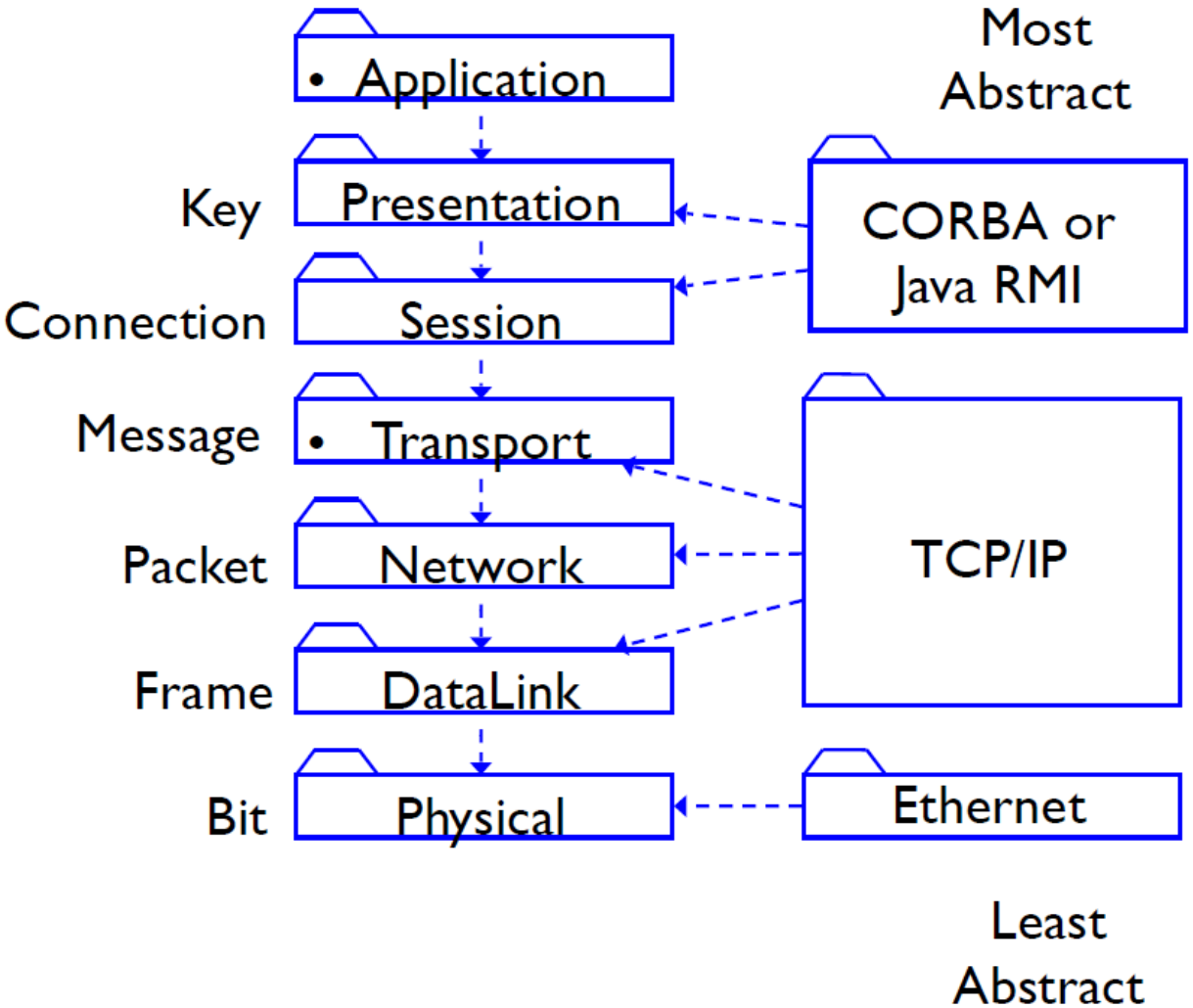
Dividing a system into independent peer subsystems

- subsystems provide services to each other on same level of abstraction
- each subsystem responsible for a different class of services

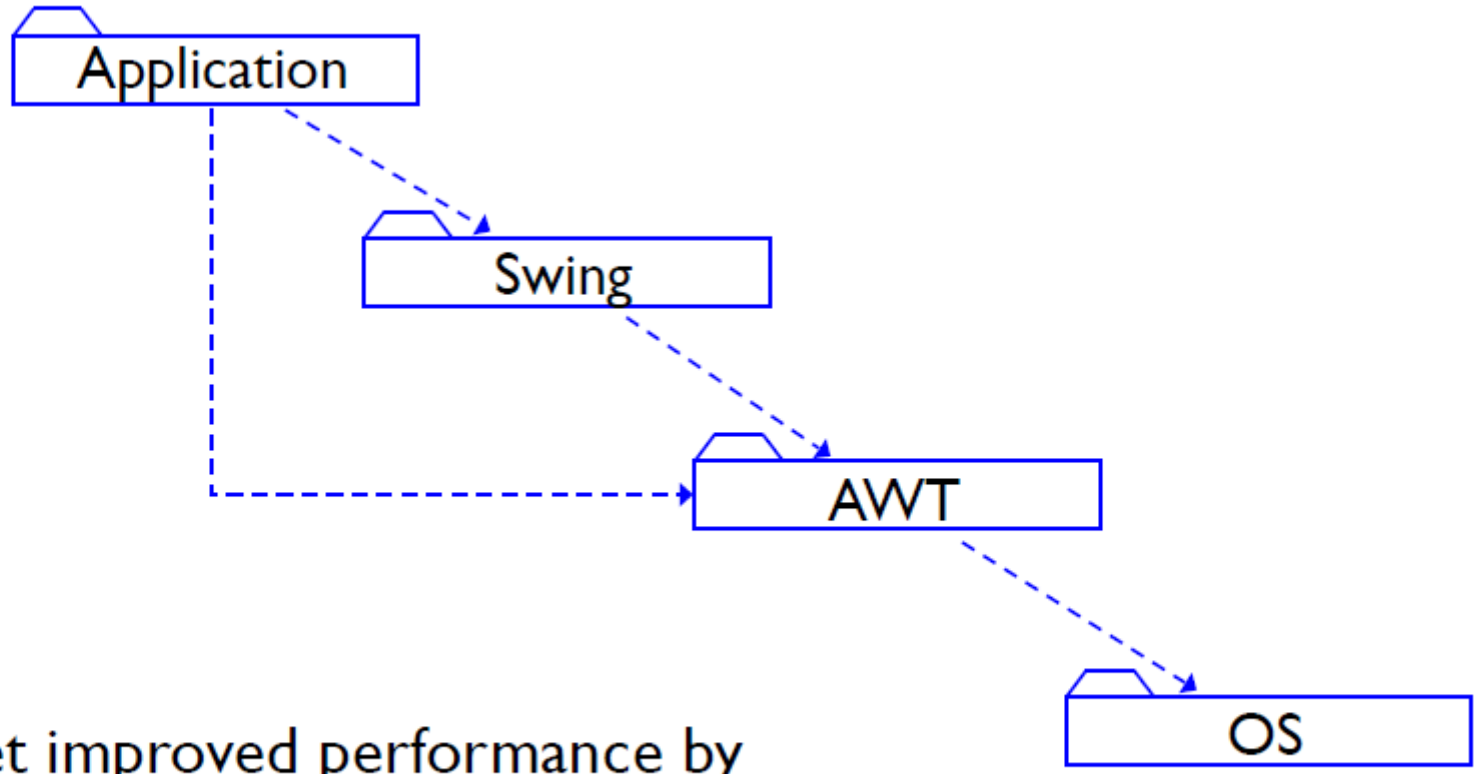
Subsystem decomposition often involves <u>both</u> partitioning and layering

Example: OSI Model

- Stands for "Open Systems Interconnection"

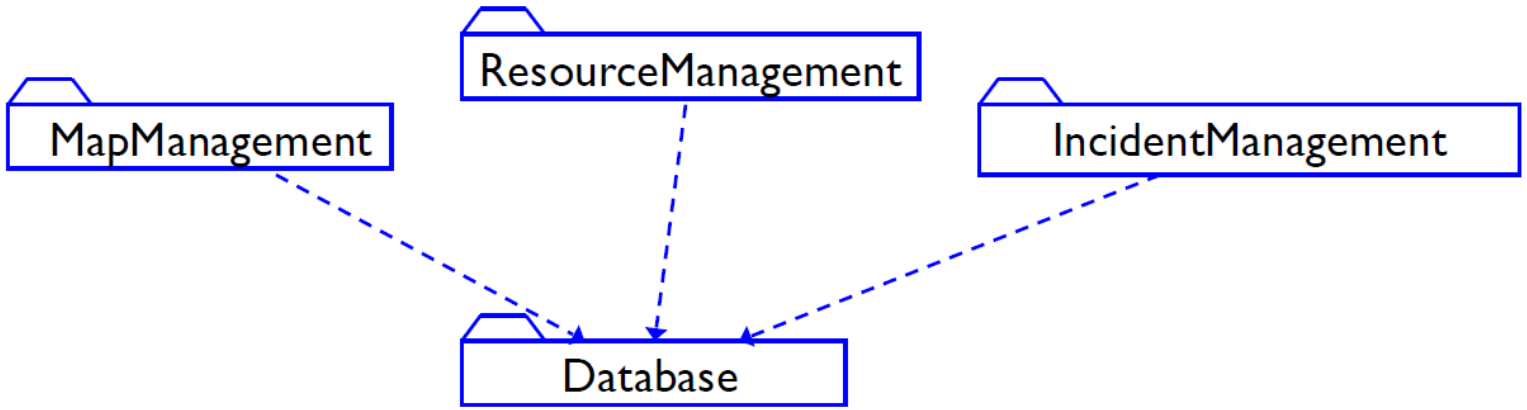


Example: Java Swing - Open or closed?



- You can get improved performance by bypassing the Swing layer.

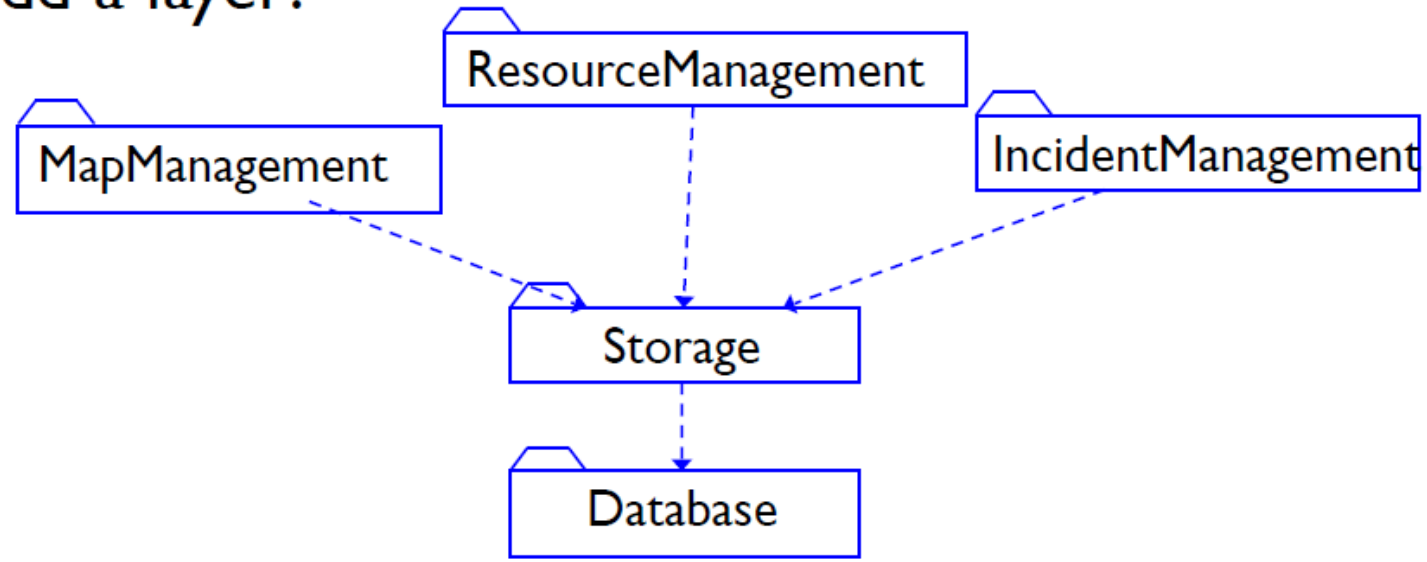
Example: The emergency response database system



- The Database subsystem is directly coupled to the other three subsystems, making these subsystems vulnerable to changes in Database design.

Example: The emergency response database system

- Add a layer:



- Storage will have a more stable interface.
  - If Database changes then only Storage will have to be modified. Less coupling to Database!

Design Goals

- Are taken from the existing non-functional requirements and from your observations and documentation of the application domain.

- These goals usually lie in the following categories:
  - Performance
  - Dependability
  - Cost
  - Maintenance
  - End User Criteria

Design Goals – Performance

- Response Time
  - How quickly must the system respond to a user request?
- Throughput
  - How many tasks does the system need to complete in a fixed period of time?
- Memory
  - How much memory is required when running?

Design Goals – Dependability

- Robustness
  - Survive invalid user input.
- Reliability
  - Does observed behaviour match specified behaviour?
- Availability
  - Percentage of time the system is available to perform normal tasks.
- Fault Tolerance
  - Ability to continue to operate under erroneous conditions.

Design Goals – Dependability, Cont.

- Security
  - Ability to withstand malicious attacks.

- Safety
  - Avoid human injury even in the presence of errors and failures.

Design Goals – Cost

- Development Cost
  - Initial system development.
- Deployment Cost
  - Cost of installation and user training.
- Upgrade Cost
  - Translating data from previous system. Backwards compatibility?
- Maintenance Cost
  - Cost of bug fixes and enhancements.
- Administration Cost
  - Cost to administer system.

Design Goals – Maintenance

- Extensibility
  - How easy is it to add functionality or new classes to the system?
- Modifiability
  - How easy is it to change functionality?
- Adaptability
  - How easy is to port the system to different application domains?
- Portability
  - How easy is it to port the system to different platforms?

Design Goals – Maintenance, Cont.

- Readability
  - How easy is it to understand the system from reading the code?

- Traceability of Requirements
  - How easy is it to map the code to specific requirements?

Design Goals – End User Criteria

- Utility
  - How well does the system support the work of the user?

- Usability
  - How easy is it for the user to use the system?

Trade-Offs!

- Of course these goals can beat on each other.

- For example it might not be possible to have a system that is completely safe and secure but still cheap.

- For example, you might have to compromise between:
  - Space and Speed
  - Delivery Time and Functionality
  - Delivery Time and Quality
  - Delivery Time and Staffing

Boundary Conditions

- Might already have boundary Use Cases – but not always.

- Configuration Use Cases:
  - First, identify each persistent object.
  - Second, locate the use case that creates the object and then the use case that archives the object. If these cannot be found, create the necessary use cases invoked by a system administrator.

Boundary Conditions, Cont.

- Start-up and Shut-down Use Cases:

- Each component (ex. *WebServer*) will need a start, shutdown and configure use case.

- A single use case might manage several tightly coupled components.

Boundary Conditions, Cont.

- Exception Handling Use Cases:

- Identify each possible component failure (ex. *NetworkOutage*), and decide how the system should react.

- Exceptions can be generated by hardware failures, changes in the operating environment, and software faults.

- How to handle these exceptions to prevent failure is a big topic!

Handling Boundary Conditions

- These new boundary use cases could have a big impact on your design and you might have to go back and change your system decomposition to account for them.

- But it is still easier to look at boundary conditions after you have built your basic design.

Next…

Testing